

Internship report

Djamila Mohamed

Supervisor: Adel Djoudi, Formal Methods Expert

Referent Professor: Ahmed Bouajjani

April 4, 2024



Contents

1	The company	2
1.1	DIS_IBS branch	2
1.2	Organization chart	2
2	Topic of the internship	2
3	Study of a solution for code editor users	3
3.1	Background	3
3.2	Language Servers	4
3.2.1	Improved Performance	4
3.2.2	Language Agnostic	4
3.2.3	Centralized Maintenance	4
3.2.4	Extensibility	5
3.2.5	Examples of language servers	5
4	Frama-C plugin development	5
5	Future work	5

1 The company

1.1 DIS_IBS branch

1.2 Organization chart

text

2 Topic of the internship

Java card, EAL certif etc.

My role as an intern at Thales DIS is to develop a code assistance tool for Frama-C developers who use ACSL annotations in their C source code. ACSL (ANSI/ISO C Specification Language) is a language used to specify the behaviour of programs written using the C language, it is used to ensure the correctness of functions

and verify their behaviour corresponds to their specification. [1]

3 Study of a solution for code editor users

3.1 Background

Developers often struggle to choose the right development environment for their software projects. A lot of editors are greedy in storage capacity and often specialize their tools around few programming languages, which are generally the most popular ones. Let's take for example JetBrains' IntelliJ IDEA (Community edition), we are indeed well served with all the Java tooling and libraries, but we will not be able to start Javascript projects or have other Java framework support without paying, and IntelliJ only supports few languages [2], same goes for other well-known editors. Developers want a versatile and lightweight IDE (Integrated Development Environment) with the necessary code assisting tools where they can find all the code comprehension and coding features they can find in regular editors, such as real time code completion help, finding where a method or a variable is defined, displaying information about a symbol when hovering it with the cursor, etc. On top of this, such features should be displayed to the user within a reasonable amount of time. A solution was recently found in order to make up for these shortcomings, which is based on a client and server communication between the editor and a language server.

3.2 Language Servers

Language Servers [3] come in handy in our situation. The language server protocol (LSP) is a standardized way for editors and IDEs to communicate with language servers. This protocol defines a set of standard methods and data structures that language servers must implement, allowing them to be integrated with a wide range of development tools. By using a language server, editors and IDEs can offload the language-specific processing tasks to the server, which can then provide the necessary code comprehension and coding features to the client in a timely and efficient manner. This approach has several advantages:

3.2.1 Improved Performance

Language servers can leverage the processing power of a dedicated server to perform complex language analysis tasks, such as code completion, symbol lookup, and hover information, without burdening the editor or IDE. This results in faster response times and a more responsive development experience for the user.

3.2.2 Language Agnostic

Language servers are designed to be language-agnostic, meaning that a single language server can support multiple programming languages. This allows developers to use their preferred editor or IDE with a wide range of languages, without having to switch between different development environments.

3.2.3 Centralized Maintenance

Language servers are maintained and updated independently of the editor or IDE, making it easier to keep the language-specific features up-to-date. Developers can benefit from the latest improvements and bug fixes without having to update their entire development environment.

3.2.4 Extensibility

Language servers can be extended to support additional features or customized to meet the specific needs of a development team or project. This allows for a more tailored development experience that aligns with the team's workflows and preferences. See [4], [5], [6] for more information about LSP.

3.2.5 Examples of language servers

A recent example is the Nickel Language Server developed by Yannik Sander under the Tweag company (developed the Nickel configuration language)

4 Frama-C plugin development

5 Future work

6 Bibliography